

Medium → Advanced Java Developer Masterclass

Complete Technical Guide: Core Java, Concurrency, Spring Boot, Kafka, Algos, Diagrams & Code

Target: Senior Java Developer Masterclass | Prepared by: Shujat Khan

MASTER INDEX OF INTERVIEW TOPICS

#	Topic / Question Area	Category
1	Composition vs Inheritance	Core Java & OOPs
2	JVM ClassLoader Hierarchy	Core Java & OOPs
3	Immutability & Designing Immutable Classes	Core Java & OOPs
4	Fail-Fast vs Fail-Safe Iterators	Core Java & OOPs
5	equals() and hashCode() Effects on HashMap	Core Java & OOPs
6	Reflection API (Pros & Cons)	Core Java & OOPs
7	Callable vs Runnable	Core Java & OOPs
8	Detecting & Fixing Memory Leaks	Core Java & OOPs
9	Java Memory Model & Happens-Before Ordering	Multithreading & Concurrency
10	synchronized vs Lock vs Atomic Classes	Multithreading & Concurrency
11	ConcurrentHashMap Internal Thread-Safety	Multithreading & Concurrency
12	Thread Starvation vs Livelock vs Deadlock	Multithreading & Concurrency
13	CompletableFuture for Async Programming	Multithreading & Concurrency
14	ForkJoinPool vs ExecutorService	Multithreading & Concurrency
15	Semaphore vs CountdownLatch vs CyclicBarrier	Multithreading & Concurrency
16	ThreadLocal Usage & Pitfalls	Multithreading & Concurrency
17	Spring Boot Auto-Configuration Internals	Spring Boot & Microservices
18	Stereotype Annotations (@Component, @Service, etc.)	Spring Boot & Microservices
19	@Transactional Propagation & Isolation Levels	Spring Boot & Microservices
20	Idempotency in REST APIs	Spring Boot & Microservices
21	API Gateway Responsibilities	Spring Boot & Microservices
22	Circuit Breaker vs Retry Pattern	Spring Boot & Microservices
23	Saga Pattern vs Two-Phase Commit (2PC)	Spring Boot & Microservices
24	Securing Microservices (JWT, OAuth2)	Spring Boot & Microservices
25	Kafka Producer ACKs (0, 1, all)	Kafka & Event-Driven Systems
26	Kafka Consumer Groups & Partition Rebalancing	Kafka & Event-Driven Systems
27	Kafka Ordering & Durability Guarantees	Kafka & Event-Driven Systems

28	Kafka Log Compaction	Kafka & Event-Driven Systems
29	Exactly-Once Semantics in Kafka	Kafka & Event-Driven Systems
30	Kafka vs RabbitMQ vs Pulsar	Kafka & Event-Driven Systems
31	Kafka Streams Real-Time Processing Use Cases	Kafka & Event-Driven Systems
32	Kafka High Throughput Architecture at Scale	Kafka & Event-Driven Systems

DETAILED EXPLANATIONS, ALGORITHMS, FLOWCHARTS & JAVA PROGRAMS

1. Composition vs Inheritance

Explanation: Inheritance ("is-a") tightly couples classes and breaks encapsulation if superclass internals change. Composition ("has-a") delegates behavior to inner objects, promoting flexibility and runtime behavior modification.

Algorithm: 1. Identify shared behaviors. 2. Instead of extending a base class, declare interface types as private fields. 3. Delegate method calls to component instances.

Inheritance: [BaseClass] <--- [SubClass] (Tightly Coupled)
Composition: [ContainerClass] has-a [ComponentInterface] -> [ConcreteComponent] (Flexible)

```
interface Engine { void start(); }
class V8Engine implements Engine {
    public void start() { System.out.println("V8 Engine started."); }
}
class Car {
    private final Engine engine; // Composition
    public Car(Engine engine) { this.engine = engine; }
    public void drive() { engine.start(); }
}
```

2. JVM ClassLoader Hierarchy

Explanation: Classes are loaded hierarchically via Delegation Model: Bootstrap ClassLoader -> Platform/Extension ClassLoader -> Application/System ClassLoader -> Custom ClassLoaders.

Algorithm: When requested to load a class, parent classloader is checked first. If parent cannot find it, current classloader attempts loading.

[Application ClassLoader] -> checks -> [Platform ClassLoader] -> checks -> [Bootstrap ClassLoader]

```
public class ClassLoaderDemo {
    public static void main(String[] args) {
        Class<?> clazz = ClassLoaderDemo.class;
        System.out.println("Loader: " + clazz.getClassLoader());
        System.out.println("Parent: " + clazz.getClassLoader().getParent());
    }
}
```

3. Immutability & Designing Immutable Classes

Explanation: An immutable class has state that cannot change after construction. Rules: 1. Make class `final` (or private constructors). 2. Make all fields `private` and `final`. 3. Do not provide setter methods. 4. Deep copy mutable reference fields in constructors and getters.

Algorithm: Initialize all properties strictly via constructor parameters. Clone mutable objects (like Date or Lists) before assignment.

[Immutable Object Construction] ---> [Fields Final & Private] ---> [No Setters] ---> [Thread Safe State]

```
import java.util.Date;
public final class ImmutablePerson {
    private final String name;
    private final Date birthDate;
    public ImmutablePerson(String name, Date birthDate) {
        this.name = name;
        this.birthDate = new Date(birthDate.getTime()); // Defensive copy
    }
    public String getName() { return name; }
    public Date getBirthDate() { return new Date(birthDate.getTime()); } // Defensive copy
}
```

4. Fail-Fast vs Fail-Safe Iterators

Explanation: Fail-fast iterators (e.g., `ArrayList` iterator) throw `ConcurrentModificationException` if collection is structurally modified during iteration. Fail-safe iterators (e.g., `ConcurrentHashMap` iterator) operate on a clone/snapshot of data.

Algorithm: Fail-fast checks modCount against expectedModCount on every `next()` call. If mismatch, throws exception.

[Collection ModCount Check] ---> Match? ---> Continue OR Throw ConcurrentModificationException

```
import java.util.*;
public class FailFastDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(Arrays.asList("A", "B"));
        try {
            for (String s : list) {
                if (s.equals("A")) list.remove(s); // Triggers ConcurrentModificationException
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("Caught Fail-Fast Exception!");
        }
    }
}
```

5. equals() and hashCode() Effects on HashMap

Explanation: `hashCode()` determines bucket index location. `equals()` resolves bucket collisions by comparing object equality. Broken contracts cause key duplication or search failures.

Algorithm: 1. Hash key. 2. Compute bucket index. 3. Check bucket head hash. 4. Iterate collision list comparing `equals()`.

[Key] -> hashCode() -> [Bucket Index] -> collision check via equals() -> [Value]

```
import java.util.Objects;
public class CustomKey {
    private final String id;
    public CustomKey(String id) { this.id = id; }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        CustomKey key = (CustomKey) o;
        return Objects.equals(id, key.id);
    }
    @Override public int hashCode() { return Objects.hash(id); }
}
```

6. Reflection API in Java (Pros & Cons)

Explanation: Allows inspection and execution of classes, methods, and fields at runtime. Pros: Framework extensibility (Spring/Hibernate). Cons: Performance overhead, security manager restrictions, breaks encapsulation.

Algorithm: Access class metadata via `Class.forName()`, invoke private methods using `setAccessible(true)`.

[Runtime Class Loading] ---> [Inspect Metadata] ---> [Bypass Encapsulation via setAccessible(true)]

```
import java.lang.reflect.Method;
public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        Class<?> c = Class.forName("java.util.ArrayList");
        Method m = c.getMethod("size");
        System.out.println("Method found: " + m.getName());
    }
}
```

7. Callable vs Runnable

Explanation: `Runnable` run method returns `void` and cannot throw checked exceptions. `Callable` call method returns a generic result (`V`) and can throw checked exceptions.

Algorithm: Submit `Callable` to `ExecutorService`, which returns a `Future` object to retrieve asynchronous results.

[Callable Task] ---> submitted to Executor ---> Returns [Future] object

```
import java.util.concurrent.*;
public class CallableDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newSingleThreadExecutor();
        Callable<String> task = () -> "Callable Result";
        Future<String> future = ex.submit(task);
        System.out.println(future.get());
        ex.shutdown();
    }
}
```

8. Detecting and Fixing Memory Leaks in Java Apps

Explanation: Memory leaks occur when objects are no longer needed but remain reachable via GC roots (e.g., static collections, unclosed streams, ThreadLocals). Fixed via profilers (VisualVM, JProfiler) and heap dump analysis.

Algorithm: 1. Capture heap dump. 2. Analyze Dominator Tree. 3. Identify leaky GC roots. 4. Fix reference retention.

[GC Roots] ---> [Static Reference / Unclosed Resource] ---> [Uncollectable Heap Memory (Leak)]

// Fix Pattern: Always remove ThreadLocal variables or unregister listeners in finally blocks

9. Java Memory Model & Happens-Before Ordering

Explanation: JMM defines multi-threaded memory visibility rules. Happens-before guarantees that memory writes by one thread are visible to subsequent reads by another (established via `volatile`, `synchronized`, or thread start/join).

Algorithm: Insert memory barriers (`LoadLoad`, `StoreStore`) to prevent CPU instruction reordering.

Thread A [Write Volatile] --(Happens-Before)--> Thread B [Read Volatile]

```
public class VolatileHappensBefore {
    private volatile boolean flag = false;
    public void writer() { flag = true; } // Establishes happens-before edge
    public void reader() { if (flag) { /* safe read */ } }
}
```

10. synchronized vs Lock vs Atomic Classes

Explanation: `synchronized` is keyword-based monitor locking. `ReentrantLock` offers fairness, try-lock, and condition variables. `Atomic classes` provide lock-free hardware-level CAS operations.

Algorithm: Atomic classes execute CPU-level `CMPXCHG` (Compare-And-Swap) loops without thread suspension.

```
[AtomicInteger.incrementAndGet()] ---> Hardware CAS Loop (Lock-Free) ---> Success
```

```
import java.util.concurrent.atomic.AtomicInteger;
public class AtomicDemo {
    private final AtomicInteger counter = new AtomicInteger(0);
    public void increment() { counter.incrementAndGet(); }
}
```

11. ConcurrentHashMap Internal Thread-Safety

Explanation: Java 8 `ConcurrentHashMap` uses CAS operations for empty bucket insertions and synchronized blocks strictly on individual bucket head nodes during hash collisions, avoiding full map locks.

Algorithm: Compute hash -> Check bucket -> If empty, atomic CAS insert -> If collision, synchronized lock on bin node.

```
[Hash Key] ---> Bucket Empty? ---> CAS Insert (Lock-Free) OR Synchronized Bin Lock
```

```
import java.util.concurrent.*;
public class ConcurrentMapTest {
    public static void main(String[] args) {
        ConcurrentMap<String, Integer> map = new ConcurrentHashMap<>();
        map.put("Key", 100);
        System.out.println(map.get("Key"));
    }
}
```

12. Thread Starvation vs Livelock vs Deadlock

Explanation: Deadlock: Threads blocked indefinitely waiting for each other. Livelock: Threads continuously change state in response to each other without making progress. Starvation: Thread is perpetually denied CPU time by greedy threads.

Algorithm: Prevent deadlock via lock ordering; prevent starvation via fair locks (`new ReentrantLock(true)`).

```
Deadlock (Blocked) vs Livelock (Active Loop) vs Starvation (Ignored by Scheduler)
```

```
// Fairness configuration: new ReentrantLock(true) ensures FIFO thread scheduling
```

13. CompletableFuture for Async Programming

Explanation: Represents asynchronous computations allowing non-blocking pipeline chaining (`thenApply`, `thenCompose`, `allOf`) across thread pools.

Algorithm: Submit task asynchronously -> Attach completion callbacks -> Return Future instantly.

[supplyAsync] ---> [thenApply] ---> [thenAccept] (Non-Blocking Pipeline)

```
import java.util.concurrent.*;
public class AsyncDemo {
    public static void main(String[] args) {
        CompletableFuture.supplyAsync(() -> "Hello")
            .thenApply(s -> s + " World")
            .thenAccept(System.out::println);
    }
}
```

14. ForkJoinPool vs ExecutorService

Explanation: `ForkJoinPool` utilizes work-stealing algorithms optimized for recursive divide-and-conquer tasks. Standard `ExecutorService` uses a centralized shared blocking queue for independent tasks.

Algorithm: Idle worker threads steal tasks from the deque of busy worker threads.

[Worker Thread Deque] <--- Work Stealing ---> [Idle Worker Thread]

```
import java.util.concurrent.*;
public class ForkJoinExample {
    public static void main(String[] args) {
        ForkJoinPool pool = ForkJoinPool.commonPool();
        pool.submit(() -> System.out.println("Running in ForkJoinPool"));
    }
}
```


15. Semaphore vs CountDownLatch vs CyclicBarrier

Explanation: `Semaphore` maintains a set of permits for resource throttling. `CountDownLatch` allows one or more threads to wait until a set of operations completes. `CyclicBarrier` allows a fixed number of threads to wait for each other at a common barrier point.

Algorithm: CountDownLatch decrements count on `countDown()`, releases waiters at zero. CyclicBarrier resets count upon tripping.

[Threads] ---> CountDownLatch (Wait for N counts) ---> Resumes execution at 0

```
import java.util.concurrent.*;
public class LatchDemo {
    public static void main(String[] args) throws Exception {
        CountDownLatch latch = new CountDownLatch(2);
        latch.countDown(); latch.countDown();
        latch.await();
        System.out.println("Latch released!");
    }
}
```

16. ThreadLocal Usage & Pitfalls

Explanation: Provides thread-isolated variables. Pitfall: In thread pools (like Tomcat), failing to call `threadLocal.remove()` causes severe memory leaks and cross-request data pollution.

Algorithm: Map thread instances to local storage entries. Always clean up in `finally` blocks.

[Thread] ---> [ThreadLocalMap] ---> [Isolated Variable Value]

```
public class ThreadLocalContext {
    private static final ThreadLocal<String> userContext = new ThreadLocal<>();
    public static void set(String user) { userContext.set(user); }
    public static void clear() { userContext.remove(); } // Crucial cleanup
}
```

17. Spring Boot Auto-Configuration Internals

Explanation: `@EnableAutoConfiguration` uses `SpringFactoriesLoader` to scan `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, evaluating conditional annotations (`@ConditionalOnClass`).

Algorithm: Scan classpath -> Evaluate conditional filters -> Instantiate matching auto-configured beans.

[@EnableAutoConfiguration] ---> [Scan Imports File] ---> [Evaluate Conditions] ---> [Load Beans]

```
// Spring Boot Core: META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

18. Stereotype Annotations (@Component, @Service, etc.)

Explanation: `@Service`, `@Repository`, and `@Controller` are meta-annotated with `@Component`. `@Repository` additionally enables automatic JDBC/JPA persistence exception translation.

Algorithm: Component scanner identifies classes annotated with stereotype markers and registers them into Spring IoC container.

```
[@Component] ---> Meta-annotated ---> [@Service / @Repository / @Controller]
```

```
import org.springframework.stereotype.Service;
@Service
public class PaymentService {
    public void process() { System.out.println("Processing payment..."); }
}
```

19. @Transactional Propagation & Isolation Levels

Explanation: Propagation defines transaction boundary rules (e.g., `REQUIRED`, `REQUIRES\_NEW`). Isolation levels (`READ\_COMMITTED`, `SERIALIZABLE`) manage concurrent transaction anomalies.

Algorithm: Spring AOP creates transaction proxy, opening/committing connections via PlatformTransactionManager.

```
[Client Request] ---> [Spring AOP Transaction Proxy] ---> [Open Connection & Commit/Rollback]
```

```
import org.springframework.transaction.annotation.*;
@Transactional(propagation = Propagation.REQUIRED, isolation = Isolation.READ_COMMITTED)
public void executeTransfer() { /* Transactional boundary */ }
```

20. Idempotency in REST APIs

Explanation: Ensures making multiple identical requests has the same effect as a single request. Implemented using client-supplied `Idempotency-Key` headers stored in Redis with result caching.

Algorithm: Check Redis for Idempotency-Key -> If exists, return cached response -> Else, execute transaction and cache result.

```
[Request + Idempotency-Key] ---> Redis Check ---> Exists? Return Cached : Execute & Cache
```

```
// Idempotency Pattern: Redis Distributed Lock + Result Caching
```

21. API Gateway Responsibilities

Explanation: Acts as single entry point handling cross-cutting concerns: routing, rate limiting, SSL termination, authentication/JWT validation, and response caching.

Algorithm: Intercept incoming client request -> Validate JWT -> Check Rate Limit -> Route to microservice cluster.

```
[Clients] ---> [API Gateway (Auth, Rate Limit, Routing)] ---> [Microservices]
```

```
// Gateway Pattern: Centralized edge routing and security enforcement
```

22. Circuit Breaker vs Retry Pattern

Explanation: Retry pattern handles transient downstream failures by repeating requests. Circuit Breaker stops calling failing services entirely when failure thresholds are crossed, failing fast to protect system resources.

Algorithm: Track failure rate -> If > threshold, trip circuit to OPEN state -> Return fallback response.

```
[CLOSED State] --(Failures > Threshold)--> [OPEN State (Fast Fail & Fallback)]
```

```
// Resilience4j: @CircuitBreaker(name="backend", fallbackMethod="fallback")
```

23. Saga Pattern vs Two-Phase Commit (2PC)

Explanation: 2PC locks distributed resources synchronously (blocking availability). SAGA executes local transactions sequentially across services with compensating rollback transactions for eventual consistency.

Algorithm: Execute Step 1 -> Execute Step 2 -> If Step 2 fails, execute compensating actions in reverse order.

```
SAGA: [Step 1] ---> [Step 2 (Fail)] ---> [Compensating Step 1 Rollback]
```

```
// Distributed Transactions: SAGA choreography/orchestration with compensating rollbacks
```

24. Securing Microservices (JWT, OAuth2)

Explanation: OAuth2 Authorization Server issues signed JWT tokens. Microservices act as Resource Servers, statelessly validating cryptographic JWT signatures without centralized DB lookups.

Algorithm: Client sends Bearer Token -> Resource Server validates JWT signature using public key -> Extract roles/claims.

```
[Client] ---> [Auth Server (JWT Issue)] ---> [Resource Server (Stateless Signature Validation)]
```

```
// Security: Spring Security OAuth2 Resource Server JWT validation filter
```

25. Kafka Producer ACKs (0, 1, all)

Explanation: `acks=0`: Fire-and-forget (zero durability). `acks=1`: Leader node acknowledgment. `acks=all` (-1): Synchronized acknowledgment from leader and all in-sync replicas (ISR) for maximum durability.

Algorithm: Configure producer property `acks=all` to guarantee message persistence across broker replicas.

```
[Producer] ---> [Broker Leader] ---> [In-Sync Replicas (ISR)] ---> [ACK Sent]
```

```
// Kafka Producer Config: props.put("acks", "all");
```

26. Kafka Consumer Groups & Partition Rebalancing

Explanation: Consumer groups share message consumption loads. Partitions are evenly assigned to consumers. Rebalancing occurs when consumers join or leave, temporarily pausing consumption.

Algorithm: Group Coordinator assigns partitions using partition assignment strategies (Range, RoundRobin, CooperativeSticky).

```
[Consumer Group] ---> Partitions assigned dynamically ---> Rebalance on consumer drop/add
```

```
// Kafka Consumer Config: group.id="order-service-group"
```

27. Kafka Ordering & Durability Guarantees

Explanation: Kafka guarantees strict message ordering *only within a single partition*. Durability is achieved by writing to disk commit logs and replicating across multiple brokers.

Algorithm: Route messages with identical partition keys (e.g., `order\_id`) to the exact same topic partition.

```
[Message with Key: Order123] ---> Hashed to Partition 2 ---> Strict FIFO Ordering
```

```
// Kafka Producer: producer.send(new ProducerRecord<>("topic", orderId, payload))
```

28. Kafka Log Compaction

Explanation: Background cleaner thread retains only the latest message value for each unique key in a topic, preventing infinite log growth for state-store changelogs.

Algorithm: Scan segment logs -> Identify duplicate keys -> Retain only highest offset message per key.

```
[Key X: Val 1] ---> [Key X: Val 2 (Latest Retained via Compaction)]
```

```
// Topic Config: cleanup.policy=compact for key-value state store topics
```

29. Exactly-Once Semantics in Kafka

Explanation: Achieved via Kafka Transactional API (`producer.beginTransaction()`, `sendOffsetsToTransaction()`, `commitTransaction()`) combined with idempotent producers (`enable.idempotence=true`).

Algorithm: Write messages atomically to output topic and commit consumer offsets in a single atomic transaction.

```
[Begin Tx] ---> [Produce Messages] ---> [Commit Offsets] ---> [Atomically Committed]
```

```
// Kafka EOS: enable.idempotence=true & isolation.level=read_committed
```

30. Kafka vs RabbitMQ vs Pulsar

Explanation: Kafka: High-throughput log-based streaming, persistent partitions. RabbitMQ: Traditional message broker, complex routing (AMQP), low latency for task queues. Pulsar: Separate compute and storage architecture with tiered storage.

Algorithm: Select Kafka for event streaming scale; RabbitMQ for complex AMQP routing task queues.

```
Kafka (Log Stream) vs RabbitMQ (AMQP Broker) vs Pulsar (Tiered Storage Pub/Sub)
```

```
// Architecture Comparison: Log persistence (Kafka) vs Message routing queues (RabbitMQ)
```

31. Kafka Streams Real-Time Processing Use Cases

Explanation: Client-side stream processing library for real-time stateful transformations, windowed aggregations, and stream-table joins without external cluster overhead.

Algorithm: Read from input topic -> Apply stateless map/filter or stateful window aggregation -> Write to output topic.

```
[Kafka Input Topic] ---> [KafkaStreams Topology DSL] ---> [Kafka Output Topic]
```

```
import org.apache.kafka.streams.StreamsBuilder;
public class StreamProcessorDemo {
    public void buildPipeline(StreamsBuilder builder) {
        builder.<String, String>stream("input-topic")
            .filter((k, v) -> v.contains("ERROR"))
            .to("error-topic");
    }
}
```

32. Kafka High Throughput Architecture at Scale

Explanation: Achieved via sequential disk I/O, zero-copy memory transfers (`sendfile``), OS page cache utilization, batching, and network compression (LZ4/ZSTD).

Algorithm: Bypass JVM heap by transferring kernel page cache bytes directly to socket network buffers via zero-copy.

```
[Disk File] ---> Kernel Page Cache ---> Zero-Copy (sendfile) ---> Network Socket
```

```
// High Throughput Optimization: Zero-copy system calls and sequential disk access
```